Name: Abdur Rahman Akhtar

ID No: 94012

Program: Cyber Security

Batch No.: 056

Training Institute

Institute Name: PNY Trainings

Branch: Arfa Kareen Technology Tower

Project

Log Analysis Tool Using Python on Kali Linux

Experiment Title

Log Analysis Tool Using Python

Objective

To develop a Python-based log analysis tool that automatically parses log files, identifies suspicious activities, extracts IP addresses, and generates a security report.

Aim

- Learn how security analysts analyze log files.
- Detect failed login attempts.
- Detect warning and error messages.
- Extract IP addresses.

- Generate a summarized security report.

Software Requirements

- Kali Linux 2024.x or later
- Python 3.x
- VS Code or Nano Editor
- Terminal
- Sample log file

Hardware Requirements

- 4 GB RAM (Minimum)
- 20 GB Disk Space
- Intel/AMD Processor
- VirtualBox or VMware (Optional)

Theory

Logs are records generated by operating systems, servers, applications, and security devices. Security analysts examine these logs to detect suspicious behavior such as brute-force attacks, unauthorized logins, malware execution, and application failures.

Python simplifies log analysis by reading files, searching for patterns, extracting IP addresses using Regular Expressions (Regex), and generating reports automatically.

Procedure

Step 1: Update Kali Linux

```
sudo apt update
sudo apt upgrade -y
```

Step 2: Verify Python Installation

```
python3 --version
```

Expected Output

```
Python 3.x.x
```

Step 3: Create Working Directory

```
mkdir LogAnalysisLab
cd LogAnalysisLab
```

Step 4: Create Python File

```
nano log_analyzer.py
```

Python Program:

```
#!/usr/bin/env python3

import re
import csv
import json
from collections import Counter
from datetime import datetime

print("=" * 60)
print("     CYBER SECURITY LOG ANALYSIS TOOL")
print("=" * 60)

log_file = input("Enter log file path: ")

try:
   with open(log_file, "r", encoding="utf-8", errors="ignore") as f:
      logs = f.readlines()

except FileNotFoundError:
   print("[-] Log file not found!")
   exit()

# -------------------------
# Counters
# -------------------------

errors = 0
```

```
warnings = 0
critical = 0
failed = 0
success = 0
sudo = 0
ssh = 0

ip_pattern = r'(?:\d{1,3}\.){3}\d{1,3}'
ip_list = []

failed_ip = Counter()

# --------------------------
# Analyze Logs
# --------------------------

for line in logs:

    l = line.lower()

    if "error" in l:
        errors += 1

    if "warning" in l:
        warnings += 1

    if "critical" in l:
        critical += 1

    if "failed" in l:
        failed += 1

    if "accepted" in l or "successful" in l:
        success += 1

    if "sudo" in l:
        sudo += 1

    if "ssh" in l:
        ssh += 1

    ips = re.findall(ip_pattern, line)

    for ip in ips:
        ip_list.append(ip)

        if "failed" in l:
            failed_ip[ip] += 1
```

```
ip_counter = Counter(ip_list)

# --------------------------
# Display Report
# --------------------------

print("\n" + "=" * 60)
print("LOG ANALYSIS REPORT")
print("=" * 60)

print(f"Total Log Entries      : {len(logs)}")
print(f"Errors                 : {errors}")
print(f"Warnings               : {warnings}")
print(f"Critical Events        : {critical}")
print(f"Failed Logins          : {failed}")
print(f"Successful Logins      : {success}")
print(f"SSH Events             : {ssh}")
print(f"Sudo Commands          : {sudo}")
print(f"Unique IP Addresses    : {len(ip_counter)}")

print("\nTop 10 IP Addresses")
print("-" * 60)

for ip, count in ip_counter.most_common(10):
    print(f"{ip:20} {count}")

print("\nPossible Brute Force IPs")
print("-" * 60)

found = False

for ip, count in failed_ip.items():

    if count >= 5:
        print(f"{ip} --> {count} failed attempts")
        found = True

if not found:
    print("No brute-force activity detected.")

# --------------------------
# Save TXT Report
# --------------------------

with open("analysis_report.txt", "w") as report:

    report.write("LOG ANALYSIS REPORT\n")
```

```
    report.write("=" * 50 + "\n")

    report.write(f"Generated : {datetime.now()}\n\n")

    report.write(f"Total Entries : {len(logs)}\n")
    report.write(f"Errors : {errors}\n")
    report.write(f"Warnings : {warnings}\n")
    report.write(f"Critical : {critical}\n")
    report.write(f"Failed Logins : {failed}\n")
    report.write(f"Successful Logins : {success}\n")
    report.write(f"SSH Events : {ssh}\n")
    report.write(f"Sudo Commands : {sudo}\n")

    report.write("\nTop IP Addresses\n")

    for ip, count in ip_counter.most_common(10):
        report.write(f"{ip} : {count}\n")

# -------------------------
# Save CSV
# -------------------------

with open("analysis_report.csv", "w", newline="") as csvfile:

    writer = csv.writer(csvfile)

    writer.writerow(["Metric", "Count"])

    writer.writerow(["Total Logs", len(logs)])
    writer.writerow(["Errors", errors])
    writer.writerow(["Warnings", warnings])
    writer.writerow(["Critical", critical])
    writer.writerow(["Failed Logins", failed])
    writer.writerow(["Successful Logins", success])
    writer.writerow(["SSH Events", ssh])
    writer.writerow(["Sudo Commands", sudo])

# -------------------------
# Save JSON
# -------------------------

data = {

    "total_logs": len(logs),
    "errors": errors,
    "warnings": warnings,
    "critical": critical,
    "failed_logins": failed,
```

```
    "successful_logins": success,
    "ssh_events": ssh,
    "sudo_commands": sudo,
    "top_ips": dict(ip_counter.most_common(10))

}

with open("analysis_report.json", "w") as js:
    json.dump(data, js, indent=4)

print("\nReports Generated Successfully!")

print("analysis_report.txt")
print("analysis_report.csv")
print("analysis_report.json")
```

Save:

CTRL + O

ENTER

CTRL + X

Step 5: Create Sample Log File

```
nano sample.log
```

```
Session  Actions  Edit  View  Help
  GNU nano 8.6                                                  sample.log
2026-06-25 08:00:01 INFO System boot completed
2026-06-25 08:01:12 INFO User 'kali' logged in from 192.168.1.10
2026-06-25 08:02:15 WARNING High CPU usage detected
2026-06-25 08:03:22 ERROR Failed to start Apache service
2026-06-25 08:04:11 INFO SSH connection accepted from 192.168.1.20
2026-06-25 08:05:35 Failed password for root from 192.168.1.15 port 22 ssh2
2026-06-25 08:05:40 Failed password for root from 192.168.1.15 port 22 ssh2
2026-06-25 08:05:45 Failed password for root from 192.168.1.15 port 22 ssh2
2026-06-25 08:05:50 Failed password for root from 192.168.1.15 port 22 ssh2
2026-06-25 08:05:55 Failed password for root from 192.168.1.15 port 22 ssh2
2026-06-25 08:06:00 Accepted password for kali from 192.168.1.20 port 22 ssh2
2026-06-25 08:06:30 sudo: kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/apt update
2026-06-25 08:07:10 WARNING Disk usage above 90%
2026-06-25 08:08:15 ERROR Database connection timeout
2026-06-25 08:09:30 CRITICAL Unauthorized file modification detected
2026-06-25 08:10:10 INFO User logout
2026-06-25 08:11:25 Failed password for admin from 10.0.0.5 port 22 ssh2
2026-06-25 08:11:40 Failed password for admin from 10.0.0.5 port 22 ssh2
2026-06-25 08:12:00 Accepted password for admin from 10.0.0.5 port 22 ssh2
2026-06-25 08:13:45 ERROR Permission denied while accessing /etc/shadow
2026-06-25 08:14:15 WARNING Memory usage exceeded threshold
2026-06-25 08:15:20 INFO Backup completed successfully
2026-06-25 08:16:30 sudo: kali : USER=root ; COMMAND=/usr/bin/systemctl restart apache2
2026-06-25 08:17:00 INFO SSH session closed for 192.168.1.20
2026-06-25 08:18:10 CRITICAL Malware signature detected
```

Step 6: Execute Program

`python3 log_analyzer.py`

When prompted

`Enter log file path:`

Type

`sample.log`

Expected Output

Conclusion

The **Log Analysis Tool** was successfully developed and tested using **Python on Kali Linux**. The tool efficiently analyzed log files, identified important security events such as errors, warnings, failed login attempts, successful logins, SSH activities, and extracted IP addresses. It also generated detailed reports that help administrators and security analysts monitor system activity and detect suspicious behavior. This project demonstrated the practical use of Python for automating log analysis, reducing manual effort, and improving the speed and accuracy of security monitoring.